Prof. Dr. Stefan Leutenegger
Teaching Assistants: Tianyi Zhang, Cédric Le Gentil, and Simon Boche

Exercise 7: Recursive Estimation

Session date: 21/04/2026

Task 1: Kinematics and Camera Measurements

The differential drive robot moving in the world x-y plane is assumed with $\mathbf{u} = [\omega_l, \omega_r]^T$, where ω_l, ω_r denote the left and right wheel rotation speeds that we can see as system inputs or, more precisely, in the context of estimation, measured wheel rotation speeds. The robot has wheel radii r and a wheel base b. It is also equipped with a forward-facing 1D camera[1] with focal length f, image width W and image centre c (no distortion) that takes measurements of N landmarks (index i) $[l_{x,i}, l_{y,i}]$.

i) Assuming the state $\mathbf{x}(t) = [x_R(t), y_R(t), \theta_R(t)]^T$ as the x-y position and orientation of the robot in a fixed world frame. The state will be estimated at discrete times t_k and will be written as $\mathbf{x}_k = [x_{R_k}, y_{R_k}, \theta_{R_k}]^T$. Write down the continuous-time kinematics as well as their discretization using Euler-forward discretization. The robot frame is defined with the x-axis pointing forward and the y-axis pointing to the left, as illustrated in Fig. 1. We also assume that in the discrete-time system, the angular increments $\Delta t \omega_j$ are affected by Gaussian noises η_r and η_l of standard deviation σ_ω.

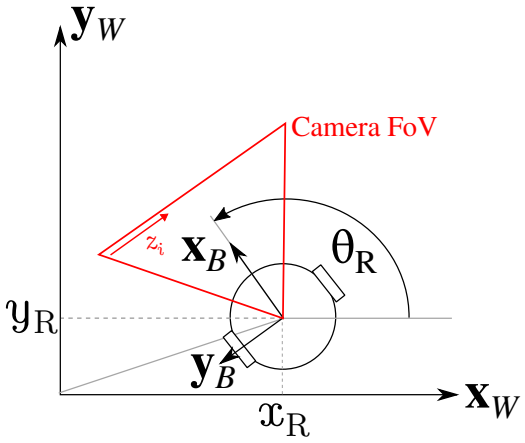

Figure 1: A two-wheel differential drive mobile robot with a front-facing 1D camera.

ii) Write down the measurement function $z_i = h_i(\mathbf{x})$ using the robot's body frame as the camera frame (with the x-axis pointing forward and the y-axis pointing to the left).

Task 2: Filter Implementations (Python)

In this task, you will implement two different recursive estimators for the above system, a Particle Filter (PF) and an Extended Kalman Filter (EKF). A simulator is provided in `E7_diff_drive_sim.py`, where you can steer the robot with the arrow keys. The simulation of (noisy) inputs $\mathbf{u}$ (which correspond to the wheel

[1] A 1D camera only collects one line of pixels. Here, we consider only a horizontal row.

rotation speeds) and landmark measurements $\tilde{z}_i$ are already implemented – the latter with a probability P to generate an outlier. Both the PF and EKF rely on the successive application of a prediction step, where the system dynamics are applied to propagate the state estimate forward in time, and an update step, where the state estimate is corrected based on the measurements. Accordingly, the script `E7_diff_drive_sim.py` provides template classes for the two filters with empty `predict(u)` and `update(z)` functions that you will implement. Note that the initialization of the filters, the sequence of steps, and some visualization functions are also already implemented.

i) Implement the localisation with an Extended Kalman Filter
(running the script with `python E7_diff_drive_sim.py ekf`).

   a) First, you will need to implement the prediction step (`predict(u)`), where you will apply the system dynamics to propagate the state estimate forward in time. This step updates both the state estimate (the mean) and the uncertainty (the covariance). After successfully implementing the prediction step, you can visualize the predicted state and uncertainty by moving the robot. You should see the uncertainty ellipse growing and the predicted state following the robot roughly, but not perfectly, since we haven't implemented the update step yet.

   b) Then, you will need to implement the update step (`update(z)`), where you will correct the state estimate based on the measurements. After successfully implementing the update step, you should see the state estimate to be much closer to the true robot pose, and the uncertainty ellipse to shrink when many landmarks are observed.

   c) It is important to note that the EKF relies on the assumption of Gaussian noise and unimodal distributions, which means that the initial state and uncertainty need to be reasonably close to the true state for the filter to converge. To illustrate this, you can try to initialize the filter with a very large uncertainty (e.g. by setting `GLOBAL_INIT = True`, and playing with the initial state/covariance values), and see how the filter performs in this case.

ii) Implement the localisation with a Particle Filter
(running the script with `python E7_diff_drive_sim.py pf`).

   a) First, you will need to implement the prediction step (`predict(u)`), where you will apply the system dynamics to propagate each particle forward in time. After successfully implementing the prediction step, you can visualize each particle moving in response to the robot's commands. You should see the particles following the robot roughly, but not perfectly, since we haven't implemented the update step yet.

   b) Then, you will need to implement the update step (`update(z)`), where you will correct the particle weights based on the measurements and resample accordingly. This corresponds to the following sub-steps:

      - Update the weights of each particle based on the likelihood of the measurement given the particle state.
      - Normalize the weights.
      - Re-sample the particles every `RESAMPLE_EVERY_N_UPDATES` updates based on the updated weights, and add some noise to the particles during re-sampling to keep diversity in the particle set (using the provided `RESAMPLING_NOISE_XY` and `RESAMPLING_NOISE_THETA` parameters).

      A few things to be careful about:

      - When projecting the landmarks into each particle's camera frame, make sure to account for the camera's field of view.
      - Make sure that the code is robust to numerical issues such as potential division by zero.

After successfully implementing the update step, you should see the particles to be much closer to the true robot pose, especially when many landmarks are observed.

c) Unlike the EKF, which provides a single state estimate, the PF provides a distribution over the state space through the particles. In a way, we have many "hypotheses" about the robot's pose. However, for downstream use, we often need to extract a single state estimate from the particle distribution. To do so, we can compute the weighted mean of the particles, which you can implement in the `getStateAndCov()` function. Be careful when computing the mean orientation, since it is a circular quantity.

d) A key advantage of the PF over the EKF is that it can represent multi-modal distributions and perform global localization, i.e. it can converge to the true state even if the initial particle distribution is randomly spread across the state space, as long as we have enough particles. To illustrate this, you can try to initialize the particles randomly across the state space (e.g. by setting `GLOBAL_INIT \ = True`), and see how the filter performs in this case. Note that the performance of the PF in this case will depend on the number of particles used. In this example, we are using a relatively small number of particles for computational reasons (naive implementation), so the filter may not always converge to the true state. In scenarios where the state/particles converge to an obviously wrong state, you can implement a simple mechanism to re-initialize the particles randomly across the state space (e.g. if none of the particles observe any of the landmarks seen by the robot for a certain number of consecutive updates).

iii) If you were to, hypothetically, implement a filter-based SLAM extension of the above, i.e. a system that also includes the landmarks in the state, briefly describe the changes that would be needed to these estimators. Would you go with a Particle Filter or an Extended Kalman Filter? Explain your choice.